

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK WORK #5.1

Threshold-Based Servo Control with FreeRTOS

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur CRAVTOV
std. gr. FAF-231

Verified by:
Alexei MARTÎNIUC
asist. univ



Chişinău, 2026



1 Objective and Technical Analysis

The objective of this laboratory work is to construct and program a smart gate control system using an Arduino Nano, an ultrasonic distance sensor, and a servo motor. The system relies on FreeRTOS to manage multiple concurrent tasks, ensuring that sensor reading, target detection, and status display remain independent and decoupled from each other.

The main learning outcomes are:

- managing task scheduling and shared states using FreeRTOS;
- processing distance measurements from an ultrasonic sensor;
- controlling a servo motor based on simple threshold ("bang-bang") logic;
- handling user inputs through a keypad and providing feedback via an I2C LCD screen;
- applying hardware abstraction layers for components (LCD, Keypad, Servo, etc.).

1.1 Specifications

The implementation follows the practical constraints in Table 1. These values capture the main components, task timing, and the safe operating range used by the control logic.

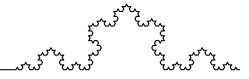
Table 1: Lab 5.1 Specifications

Item	Specification
MCU	Arduino Nano (ATmega328P)
Sensor	Ultrasonic distance sensor (TRIG/ECHO)
Actuator	Servo motor (PWM control)
Keypad	4x4 matrix keypad
Display	16x2 LCD via I2C
Control logic	Threshold with hysteresis
Task periods	Read: 100 ms, Filter: 50 ms, Display: 250 ms
Safe distance range	MIN_DISTANCE_MIN_CM to MIN_DISTANCE_MAX_CM

1.2 Threshold-Based Control

Unlike continuous PID control, this system uses threshold-based logic with hysteresis to decide the state of the gate:

- If the sensed object is closer than `minDistanceCm`, the gate opens.



- If the object recedes past `minDistanceCm + GATE_HYSTERESIS_CM`, the gate closes.

This introduces a deadband that prevents the gate from oscillating violently when the object hovers precisely on the border of the threshold.

To avoid spurious motion, the sensor reading is validated against a safe range before it can influence the gate state. If the measurement is invalid or out of range, the actuator is driven to the closed position and the internal state is reset to the default safe condition.

1.3 Sensor and Actuator Theory

The ultrasonic sensor estimates distance using time-of-flight. A short trigger pulse emits an ultrasonic burst, and the echo pulse width is proportional to the round-trip travel time. The distance is computed as:

$$d = \frac{v_{sound} \cdot t_{echo}}{2} \quad (1)$$

where v_{sound} is the speed of sound and t_{echo} is the measured echo duration. The division by 2 accounts for the outbound and return path.

The servo is controlled by a PWM signal with a typical 50 Hz frame. The pulse width determines the shaft angle, so the control logic maps the gate state into a safe position interval. Keeping a minimum and maximum bound prevents mechanical over-travel and reduces strain on the actuator.

2 System Architecture

2.1 FreeRTOS Tasks

The software is modularized into three FreeRTOS tasks to guarantee deterministic execution:

- **TaskRead (100 ms period):** Scans for keypad inputs mapped via standard I/O streams using a `KeypadStdioManager`. It accepts numeric keys to configure the threshold opening distance continuously while the system runs.
- **TaskFilter (Control Loop - 50 ms period):** Reads the `DistanceSensor`, verifies that the reading is within an acceptable range, and executes the threshold logic to adjust the `ServoActuator`. It maintains the gate's open and close states.
- **TaskDisplay (250 ms period):** Safely reads the global `sysState` variable inside a critical section and visualizes the distance, threshold configuration, servo state, and valid sensor readings onto the LCD layout.



Shared variables are updated in the control task and read in the display task inside a critical section. This prevents torn reads of distance, threshold, or gate state and keeps the LCD output consistent with the actual actuator position.

2.2 Hardware Wiring

The system features:

- An ultrasonic sensor (TRIG on A0, ECHO on D2).
- A Servo actuator on digital pin 11.
- A 4x4 matrix keypad connected across pins A1, 3-9.
- An I2C 16x2 LCD screen to output statuses and configuration values.

The Arduino Nano acts as the central controller. The ultrasonic sensor provides a digital echo pulse, the servo is driven by a PWM output, the keypad is scanned as a row/column matrix, and the LCD uses the I2C bus. This split keeps power delivery simple while the signals remain isolated and deterministic.

For reliable operation, all modules share a common ground. The servo should be powered from a stable 5V source, and the signal wires should be kept short to reduce noise pickup on the ultrasonic echo line.

Circuit Diagram - Arduino Nano Gate Controller

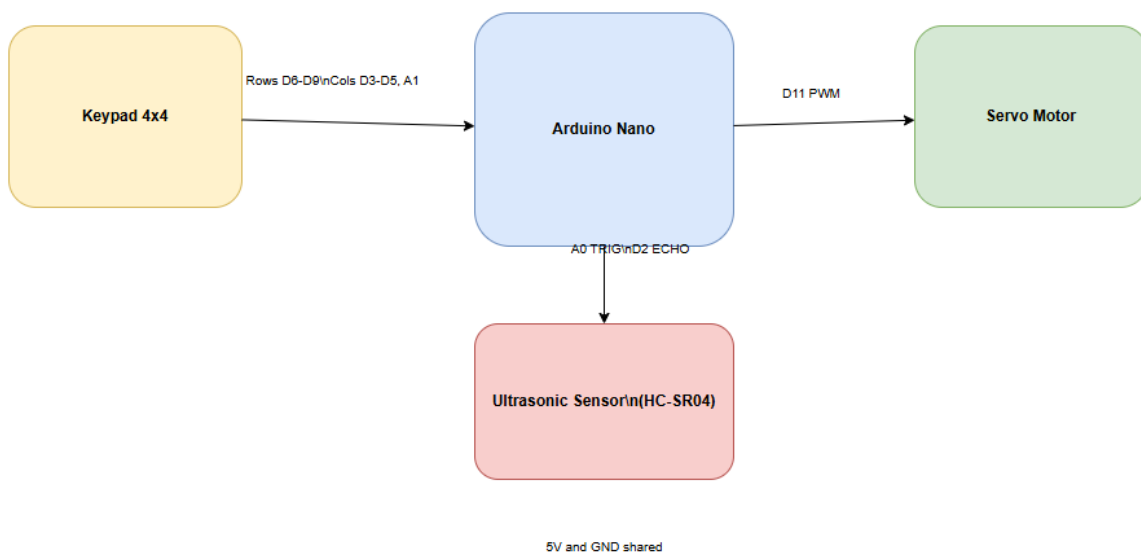
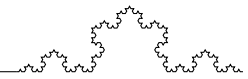


Figure 1: Improved circuit diagram with Arduino Nano, keypad, servo, and ultrasonic sensor



2.3 Architecture Diagram

The architecture diagram summarizes the data path: keypad input updates the threshold, the ultrasonic sensor provides distance feedback, the control task computes the gate state, and the display task renders status for the operator.

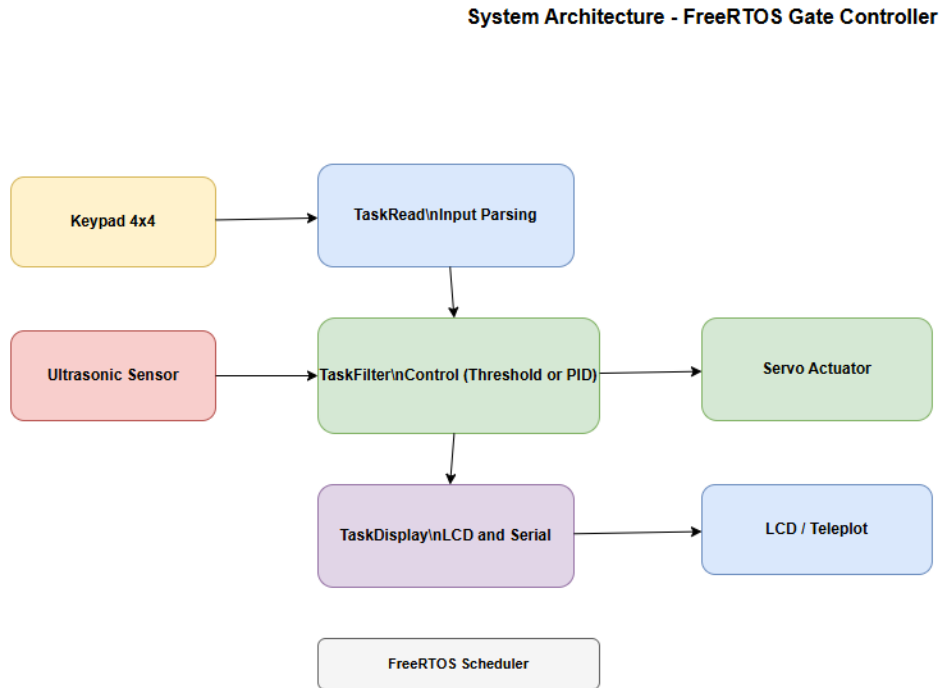


Figure 2: System architecture for the smart gate controller

2.4 Software Flow Diagram

The flow diagram highlights the periodic task execution and the hysteresis decision that keeps the gate stable near the threshold.

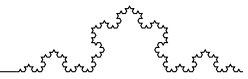
General System Flowchart — FreeRTOS Multitasking Monitor

Figure 3: Software Flow Diagram

2.5 Component Overview

2.5.1 Arduino Nano

The Arduino Nano (ATmega328P) provides a 16 MHz 8-bit CPU, 5V logic, and a limited SRAM budget. This motivates the use of compact global state and PROGMEM strings for display formatting. The board serves as the timing reference for FreeRTOS and the hub for all I/O signals.



2.5.2 Ultrasonic Sensor

The ultrasonic sensor measures distance using time-of-flight. A short trigger pulse initiates an ultrasonic burst; the echo pulse width represents the round-trip time. The reading is bounded to a safe range to avoid reacting to spurious reflections or timeouts.

2.5.3 Servo Motor

The servo receives a 50 Hz PWM signal, with pulse width encoding the target angle. The gate is only commanded inside a safe interval to avoid mechanical stalls. The threshold logic maps a valid distance condition into either a closed or open command.

2.5.4 Keypad Matrix

The 4x4 keypad is scanned as a row/column matrix. Rows are driven sequentially and columns are read to detect pressed keys. The software performs basic debouncing by sampling at a fixed period and ignores invalid characters.

2.5.5 LCD and I2C Bus

The 16x2 LCD uses the I2C bus (A4/A5), reducing the number of GPIO pins needed for the interface. The display task composes formatted lines for distance, threshold, and gate state.

2.5.6 FreeRTOS

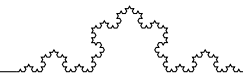
FreeRTOS provides cooperative task scheduling with deterministic delays. Separating input, control, and display logic keeps latency predictable and reduces coupling between slow I/O and time-sensitive actuator updates.

3 Conclusions

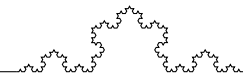
In this laboratory work, an autonomous embedded control system was implemented using FreeRTOS. Delegating input parsing, sensor logic, and visualization to separate tasks simplified timing issues and made it possible to adjust control thresholds dynamically without stuttering the physical actuator. Using hysteresis effectively prevented rapid oscillations around the threshold limits, creating a smooth and robust open/close experience.

References

- [1] Arduino Nano Documentation, <https://docs.arduino.cc/hardware/nano>



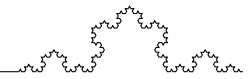
- [2] FreeRTOS API Reference, <https://www.freertos.org/a00106.html>
- [3] Arduino Servo Library Reference, <https://www.arduino.cc/reference/en/libraries/servo/>
- [4] HC-SR04 Ultrasonic Sensor Datasheet, generic module reference.
- [5] Keypad Library Reference, <https://playground.arduino.cc/Code/Keypad/>
- [6] LiquidCrystal I2C Library Reference, https://github.com/johnrickman/LiquidCrystal_I2C



4 Appendix - Source Code

4.1 main.cpp

```
1 #include <Arduino.h>
2 #include <Arduino_FreeRTOS.h>
3 #include <Keypad.h>
4 #include <Wire.h>
5 #include <LiquidCrystal_I2C.h>
6 #include <lcd/LcdStdioManager.h>
7 #include <keypad/KeypadStdioManager.h>
8 #include "actuator.h"
9 #include <serialio/serialio.h>
10 #include <sensors/distance.h>
11 #include "tasks.h"
12
13 // LCD (I2C address 0x27, 16x2)
14 static LiquidCrystal_I2C lcd(0x27, 16, 2);
15
16 // Keypad pin configuration
17 static byte rowPins[4] = {9, 8, 7, 6};
18 static byte colPins[4] = {5, 4, 3, A1};
19
20 // Keypad layout (PROGMEM via Keypad library)
21 static const char keys[4][4] = {
22     {'1', '2', '3', 'A'},
23     {'4', '5', '6', 'B'},
24     {'7', '8', '9', 'C'},
25     {'*', '0', '#', 'D'}
26 };
27
28 static Keypad keypad(makeKeymap(keys), rowPins, colPins, 4, 4);
29
30 // Servo actuator on pin 11
31 static ServoActuator actuator(11);
32 // Ultrasonic sensor from nano simulation: TRIG=A0, ECHO=D2
33 static DistanceSensor distanceSensor(A0, 2);
34 static ControlContext controlContext = {&actuator, &
    distanceSensor};
```

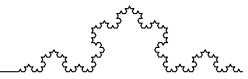



```
35
36 void setup() {
37     Serial.begin(9600);
38     actuator.begin();
39     distanceSensor.init();
40
41     lcd.init();
42     lcd.backlight();
43
44     // STDIO: LCD output, Keypad input
45     static LcdStdioManager lcdManager;
46     // lcdManager.setup(&lcd);
47     KeypadStdioManager::setup(&keypad);
48
49     redirectSerialToStdio(false, true, true);
50
51     // FreeRTOS tasks (stack sizes tuned for Nano - 2KB RAM)
52     xTaskCreate(TaskRead, "Read", 100, NULL, 1, NULL);
53     xTaskCreate(TaskFilter, "Filter", 160, &controlContext, 2,
54                 NULL);
55     xTaskCreate(TaskDisplay, "Disp", 140, NULL, 1, NULL);
56
57     // fprintf_P(stderr, PSTR("Ready\n"));
58     vTaskStartScheduler();
59 }
60 void loop() {}
```

Listing 1: Main Setup and Task Creation

4.2 tasks.cpp

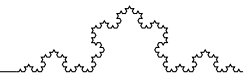
```
1 #include "tasks.h"
2 #include <Arduino.h>
3 #include <Arduino.FreeRTOS.h>
4 #include <avr/pgmspace.h>
5 #include <stdio.h>
6 #include <string.h>
7
```



```

8 // PROGMEM strings reduce SRAM usage on AVR.
9 // Each line is padded to 16 chars to fully overwrite LCD
  leftovers.
10 static const char STR_LINE0[] PROGMEM = "\rD:%3u T:%3u %c \n";
11 static const char STR_LINE0_INVALID[] PROGMEM = "\rD:—— T:%3u %
  c \n";
12 static const char STR_LINE1[] PROGMEM = "\rK:%c In:%-3s ";
13
14 // Shared state variables
15 volatile uint16_t measuredDistanceCm = 0;
16 volatile uint16_t minDistanceCm = MIN_DISTANCE_DEFAULT_CM;
17 volatile uint8_t servoAngle = 0;
18 volatile bool gateOpen = false;
19 volatile bool sensorValid = false;
20 volatile char lastKeyPressed = '-';
21 volatile SystemState sysState = {0, MIN_DISTANCE_DEFAULT_CM, 0,
  false, false, '-'};
22
23 // Input buffer for keypad threshold entry (max 3 digits: 400)
24 static char inputBuffer[4] = {0};
25 static uint8_t inputIndex = 0;
26
27 static inline void clearInputBuffer() {
28     inputIndex = 0;
29     inputBuffer[0] = '\0';
30 }
31
32 static uint16_t parseInputBuffer() {
33     uint16_t value = 0;
34     for (uint8_t i = 0; i < inputIndex; i++) {
35         value = (uint16_t)(value * 10u + (uint16_t)(inputBuffer[
          i] - '0'));
36     }
37     return value;
38 }
39
40 static inline uint16_t clampThreshold(uint16_t value) {
41     if (value < MIN_DISTANCE_MIN_CM) {

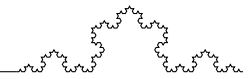
```



```

42     return MIN_DISTANCE_MIN_CM;
43 }
44 if (value > MIN_DISTANCE_MAX_CM) {
45     return MIN_DISTANCE_MAX_CM;
46 }
47 return value;
48 }
49
50 void TaskRead(void* pvParameters) {
51     (void)pvParameters;
52
53     while (true) {
54         char c = 0;
55         if (scanf("%c", &c) > 0 && c != '\0' && c != '\n' && c
56             != '\r') {
57             lastKeyPressed = c;
58             sysState.key = c;
59
60             if (c >= '0' && c <= '9') {
61                 if (inputIndex < 3) {
62                     inputBuffer[inputIndex++] = c;
63                     inputBuffer[inputIndex] = '\0';
64                 }
65             } else if (c == '#' || c == 'A') {
66                 if (inputIndex > 0) {
67                     minDistanceCm = clampThreshold(
68                         parseInputBuffer());
69                     sysState.thresholdCm = minDistanceCm;
70                     clearInputBuffer();
71                 }
72             } else if (c == '*' || c == 'B') {
73                 clearInputBuffer();
74             } else if (c == 'C') {
75                 minDistanceCm = MIN_DISTANCE_DEFAULT_CM;
76                 sysState.thresholdCm = minDistanceCm;
77                 clearInputBuffer();
78             } else if (c == 'D') {
79                 // D is a quick +5 cm step.

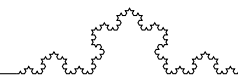
```



```

78         minDistanceCm = clampThreshold((uint16_t)(
79             minDistanceCm + 5u));
80         sysState.thresholdCm = minDistanceCm;
81         clearInputBuffer();
82     }
83
84     vTaskDelay(pdMS_TO_TICKS(TASK_READ_PERIOD));
85 }
86 }
87
88 void TaskFilter(void* pvParameters) {
89     ControlContext* ctx = static_cast<ControlContext*>(
90         pvParameters);
91     ServoActuator* actuator = ctx->actuator;
92     DistanceSensor* sensor = ctx->sensor;
93
94     actuator->setPosition(SERVO_POS_OPEN);
95     vTaskDelay(pdMS_TO_TICKS(500));
96     actuator->setPosition(SERVO_POS_CLOSED);
97     vTaskDelay(pdMS_TO_TICKS(300));
98     gateOpen = false;
99     servoAngle = actuator->getAngle();
100
101     while (true) {
102         const float rawDistance = sensor->readRaw();
103         const bool valid = (rawDistance > 0.5f) && (rawDistance
104             <= (float)MIN_DISTANCE_MAX_CM);
105         const uint16_t distanceCm = valid ? (uint16_t)(
106             rawDistance + 0.5f) : 0u;
107
108         bool nextGateState = gateOpen;
109         if (valid) {
110             const uint16_t threshold = minDistanceCm;
111             if (!gateOpen && distanceCm < threshold) {
112                 nextGateState = true;
113             } else if (gateOpen && distanceCm > (uint16_t)(
114                 threshold + GATE_HYSTERESIS_CM)) {

```



```
111         nextGateState = false;
112     }
113 } else {
114     nextGateState = false;
115 }
116
117 if (nextGateState != gateOpen) {
118     actuator->setPosition(nextGateState ? SERVO_POS_OPEN
119         : SERVO_POS_CLOSED);
119     gateOpen = nextGateState;
120 }
121
122 measuredDistanceCm = distanceCm;
123 sensorValid = valid;
124 servoAngle = actuator->getAngle();
125
126 sysState.distanceCm = measuredDistanceCm;
127 sysState.thresholdCm = minDistanceCm;
128 sysState.angle = servoAngle;
129 sysState.isOpen = gateOpen;
130 sysState.isSensorValid = sensorValid;
131
132 vTaskDelay(pdMS_TO_TICKS(TASK_CONTROL_PERIOD));
133 }
134 }
135
136 void TaskDisplay(void* pvParameters) {
137     (void)pvParameters;
138
139     while (true) {
140         uint16_t dLocal, tLocal;
141         bool openLocal, validLocal;
142         char keyLocal;
143         char localInput[4] = "____";
144
145         // capture all shared state in one critical section
146         taskENTER_CRITICAL();
147         dLocal = sysState.distanceCm;
```



```

148     tLocal = sysState.thresholdCm;
149     openLocal = sysState.isOpen;
150     validLocal = sysState.isSensorValid;
151     keyLocal = sysState.key;
152
153     if (inputIndex > 0) {
154         strncpy(localInput, inputBuffer, 3);
155         localInput[3] = '\0';
156     }
157     taskEXIT_CRITICAL();
158
159     // sanitize key display
160     if (keyLocal < 32 || keyLocal > 126) {
161         keyLocal = '-';
162     }
163
164     const char gateChar = openLocal ? 'O' : 'C';
165
166     // print line 0: distance and threshold
167     if (validLocal) {
168         printf_P(STR_LINE0, dLocal, tLocal, gateChar);
169     } else {
170         printf_P(STR_LINE0_INVALID, tLocal, gateChar);
171     }
172
173     // print line 1: key and buffer
174     printf_P(STR_LINE1, keyLocal, localInput);
175
176     // force newline to flush buffer clearly
177     printf_P(PSTR("\n\n"));
178
179     vTaskDelay(pdMS_TO_TICKS(TASK_DISPLAY_PERIOD));
180 }
181 }

```

Listing 2: FreeRTOS Task Implementations